PROJECT REPORT ON

MEDRED

MEDICINE DONATION AND DISTRIBUTION PLATFORM

"Don't waste medicines, redistribute hope."

PROJECT REPORT & TECHNICAL GUIDE

Submitted in partial fulfillment of the requirements for the award of **Diploma in Computer Engineering.**

SUBMITTED BY

MANADI RUPA	**24004-CS-098**
DASI CHARAN KUMAR	**24004-CS-080**
VOLLALA SHANMUKHA SAI	**24004-CS-117**
BEJUGAM RAGHAVI	**24004-CS-105**
MEDIPELLI SHIVANI	**24004-CS-101**
DARAVATH BASANTHI	**24004-CS-123**
BUDDA CHAITANYA	**24004-CS-068**
NALABOLU SATWIK REDDY	**24004-CS-082**
BIJILI SAI CHARAN	**24004-CS-121**

Under the guidance of

S. Ravinder, M.Tech

and Principal

P. Srinivas, M.Tech

DEPARTMENT OF COMPUTER ENGINEERING
GOVERNMENT POLYTECHNIC WARANGAL

Warangal, Telangana • Academic Session 2024–2027

DEPARTMENT OF COMPUTER ENGINEERING

GOVERNMENT POLYTECHNIC WARANGAL

CERTIFICATE

This is to certify that this project report entitled **"MEDRED: MEDICINE DONATION AND DISTRIBUTION PLATFORM"** is a bonafide record of the project work carried out by the students listed below, in partial fulfillment of the requirements for the award of Diploma in Computer Engineering, during the academic session 2024–2027.

MANADI RUPA — 24004-CS-098

DASI CHARAN KUMAR — 24004-CS-080

VOLLALA SHANMUKHA SAI — 24004-CS-117

BEJUGAM RAGHAVI — 24004-CS-105

MEDIPELLI SHIVANI — 24004-CS-101

DARAVATH BASANTHI — 24004-CS-123

BUDDA CHAITANYA — 24004-CS-068

NALABOLU SATWIK REDDY — 24004-CS-082

BIJILI SAI CHARAN — 24004-CS-121

The work is submitted for academic review. The system is a web application prototype; its results should be treated as an informational aid and not as a substitute for professional medical or commercial advice.

PROJECT GUIDE	**PRINCIPAL**
S. Ravinder, M.Tech	P. Srinivas, M.Tech

ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide, **S. Ravinder, M.Tech**, for his guidance, valuable suggestions and encouragement throughout the development of this project. His support helped us understand the technical and practical challenges involved in building a complete medicine donation and distribution platform.

We are thankful to our Principal, **P. Srinivas, M.Tech**, for providing the opportunity and institutional support to undertake this work. We also thank the faculty members of the Department of Computer Engineering for their assistance and feedback.

We acknowledge the open-source technologies used in the MEDRED project, including PHP, PostgreSQL, Supabase, Bootstrap, Razorpay, Leaflet and Playwright. Finally, we thank our families and friends for their encouragement.

PROJECT TEAM

MANADI RUPA	**24004-CS-098**
DASI CHARAN KUMAR	**24004-CS-080**
VOLLALA SHANMUKHA SAI	**24004-CS-117**
BEJUGAM RAGHAVI	**24004-CS-105**
MEDIPELLI SHIVANI	**24004-CS-101**
DARAVATH BASANTHI	**24004-CS-123**
BUDDA CHAITANYA	**24004-CS-068**
NALABOLU SATWIK REDDY	**24004-CS-082**
BIJILI SAI CHARAN	**24004-CS-121**

ABSTRACT

MEDRED is a web-based medicine donation and distribution platform that connects verified medicine distributors and pharmacies with non-governmental organizations, medical shops, old-age homes, orphanages and charitable hospitals. A large quantity of usable medicine is wasted every year because distributors hold surplus or near-expiry stock while social organizations struggle to obtain medicines for the people they serve. MEDRED, whose guiding line is “Don't waste medicines, redistribute hope,” addresses this gap with a single, verified marketplace.

The application provides role-based access for administrators, distributors and organizations. Distributors publish stock with batch number, expiry date, price and quantity. Organizations search the store, add medicines to a cart, choose a delivery address, enter a delivery mobile number and pay through Razorpay (UPI, card or net banking) or cash on delivery. Every order passes through a controlled lifecycle: Pending Payment, Pending, Accepted, Packed, Shipped, Out for Delivery and Delivered. Stock is reserved when an order is placed and restored when it is cancelled or rejected. Expired and out-of-stock medicines are removed automatically and the distributor is notified.

The system is implemented with PHP 8 on Apache, PostgreSQL hosted on Supabase, Supabase Storage for uploaded documents, Bootstrap 5 and vanilla JavaScript, and is packaged as a Docker image for deployment on Render. It includes admin verification of documents with complete removal of rejected registrations, in-app notifications with an unread badge, reviews, reports with CSV export that exclude soft-deleted medicines, audit logs, persistent thirty-day sign-in, an SMTP email-OTP password-reset flow, address-first delivery navigation and performance optimizations for a hosted database. The platform was tested at the API, browser and live-deployment levels; all functional tests passed and cached read paths respond in milliseconds.

KEYWORDS

Medicine donation • Medicine redistribution • E-commerce • Order management • Razorpay • Supabase • PHP • PostgreSQL

INDEX

CONTENTS	PAGE
Executive Summary	6
1. Introduction	7
2. Problem Statement & Motivation	8
3. Project Scope & Objectives	9
4. System Architecture	10
5. Technical Stack	12
6. Database Design	13
7. Implementation	15
8. Order Lifecycle & Payments	18
9. Security	19
10. Performance Optimization	20
11. Testing & Results	21
12. Limitations & Future Work	23
13. Conclusion & References	24
Appendix — Setup & API Reference	25

Executive Summary

MEDRED is a complete web application for redistributing usable medicines from verified distributors to verified social organizations. It replaces informal phone and spreadsheet coordination with a single platform that shows live stock, expiry dates, prices and order status.

The platform has three roles. Distributors manage their medicine inventory. Organizations browse the store, place orders and pay online or choose cash on delivery. Administrators verify accounts, review uploaded documents and monitor reports and audit logs.

The main outcomes of the project are:

- a verified network of distributors and recipient organizations;
- expiry-aware inventory with automatic removal of expired and out-of-stock medicines;
- an order lifecycle with stock reservation, cancellation, rejection and delivery tracking;
- server-verified Razorpay payments with UPI, card and net-banking support, plus cash on delivery;
- in-app notifications with an unread badge;
- reviews, reports with CSV export and a complete audit trail;
- reports that count only active medicines, so soft-deleted stock no longer inflates admin totals;
- a delivery mobile number saved with every order and reused for navigation;
- address-first navigation through Google Maps, with stored coordinates as a fallback;
- a thirty-day persistent sign-in with a signed, HttpOnly remember cookie;
- an SMTP email-OTP password-reset flow;
- rejection of pending registrations that removes the account, profile and uploaded document;
- Docker-based deployment on Render with Supabase PostgreSQL and Storage;
- caching and lazy database access that keep cached reads in the millisecond range.

The system was validated with API tests, browser tests, a role-based smoke test and live checks on the Render deployment. All functional checks passed.

1. Introduction

1.1 Background

Medicines are essential, time-sensitive products. Distributors and pharmacies routinely carry surplus stock, and a portion of it approaches expiry before it can be sold. At the same time, NGOs, old-age homes, orphanages, charitable hospitals and small medical shops often lack the funds and supply network to obtain even basic medicines. Much of the surplus is discarded while genuine demand remains unmet.

Most redistribution today happens through phone calls, messaging applications and personal contacts. These channels do not provide verified participants, live stock information, price transparency, order tracking, payment records or an audit trail. MEDRED was built to replace this informal process with a structured digital platform.

1.2 Purpose of the Project

The purpose of the project is to design and implement a reliable, easy-to-use platform where verified distributors can list surplus or near-expiry medicines and verified organizations can order them. The project also demonstrates a complete software development cycle: requirements, design, implementation, testing and deployment.

1.3 Scope of the Report

This report describes the problem, the proposed solution, the system architecture, the database design, the implementation of each module, the security and performance measures, and the testing results. Setup instructions and an API reference are included in the appendix.

2. Problem Statement & Motivation

There is no common, trustworthy platform where verified distributors can list surplus or near-expiry medicines and verified social organizations can order them. The absence of such a platform causes avoidable wastage of usable medicines, difficulty for organizations in finding suppliers, no verification of the parties involved, no record of donations, payments or deliveries, no visibility of expiry dates and stock levels, and manual, error-prone order handling.

The motivation for the project comes from three observations:

- medicine waste is often a logistics and visibility problem rather than a shortage of demand;
- social organizations need a simple ordering experience, not a complex enterprise system;
- trust is essential, so every distributor and organization must be verified before participating.

MEDRED addresses these issues with verified accounts, a live store, a controlled order lifecycle, online payments and an audit trail.

3. Project Scope & Objectives

3.1 Objectives

1. Build a verified network of medicine distributors and social organizations.
2. Provide real-time visibility of medicine stock, price, batch number and expiry date.
3. Allow organizations to search, filter, order and pay for medicines online.
4. Track every order through a clear lifecycle with cancellation and rejection support.
5. Notify distributors and organizations about orders, payments, stock and expiry events.
6. Give administrators tools to verify documents, manage users, view reports and audit activity.
7. Protect accounts and business data with authentication, authorization and secure coding practices.
8. Keep the application responsive despite a hosted database by caching read-heavy responses.

3.2 In Scope

Registration and document upload, admin verification, medicine inventory, store search and filters, cart, checkout, Razorpay and cash-on-delivery payments, order status management, cancellation and rejection, notifications, reviews, reports and CSV export, audit logs, profile management, a public map and password reset.

3.3 Out of Scope

Direct patient prescriptions, clinical advice, courier integration, a live SMS gateway, a native mobile application, multi-language support and automated document verification are outside the current scope and are listed as future work.

4. System Architecture

MEDRED uses a three-tier architecture. The browser renders static pages and JavaScript modules. The application tier is a set of PHP endpoints running under Apache in a Docker container. The data tier is a PostgreSQL database hosted on Supabase, with Supabase Storage for uploaded license and registration documents. External services provide payments and email OTP delivery.

```
Browser (HTML + Bootstrap + JavaScript)
        |
        |  JSON over HTTP
        v
Apache + PHP 8 API (Docker on Render) ---> Razorpay API (orders, payments)
        |                              ---> SMTP server (password-reset OTP)
        |                              ---> Supabase Storage (documents)
        |
        |  PDO (prepared statements)
        v
PostgreSQL database (Supabase)
```

4.1 Modules

- **Authentication and verification:** registration, login, logout, thirty-day persistent sessions, document review, account approval and complete removal of rejected registrations.
- **Medicine inventory:** add, edit, remove, stock, expiry, batch, category, price and images.
- **Store and cart:** search, filters, cart persistence, delivery address, delivery mobile number and geolocation.
- **Order management:** order creation, grouping by distributor, status transitions, cancellation, rejection and delivery navigation.
- **Payments:** Razorpay order creation and signature verification, cash on delivery and payment status.
- **Notifications:** in-app notifications, unread badge and periodic refresh.
- **Reviews:** one review per delivered order with rating and comment.
- **Reports and audit:** revenue and order statistics that ignore soft-deleted medicines, CSV export and activity log.
- **Password reset:** SMTP email OTP, reset token and password update.
- **Profile and map:** profile updates, delivery location and a public map page with address-first directions.

4.2 Design Principles

- Role-based access is enforced on the server, not only in the interface.
- Every write that affects stock runs inside a database transaction.
- Payment confirmation depends on a verified gateway signature.
- Read-heavy responses are cached briefly and invalidated on writes.
- Removed medicines are soft-deleted so historical orders remain intact, while reports and admin counts filter them out.
- The delivery address is the primary navigation target; stored coordinates are only a fallback.
- Account, document and session secrets stay in server-side environment variables.

4.3 Deployment

The application is packaged with a `Dockerfile` based on `php:8.2-apache`, with the required PDO PostgreSQL, cURL and fileinfo extensions enabled. Docker Compose provides a local development option. Production runs as a Docker web service on Render, connected to Supabase PostgreSQL and Storage. Configuration is supplied through environment variables, while `config/secrets.php` remains a local-only fallback. A lightweight `health.php` endpoint serves as the platform health check.

Uploaded documents are stored in Supabase Storage in production and on the local filesystem during development. The storage layer exposes the same helpers in both modes, so registration, verification and rejection behave identically. Render supplies HTTPS, which enables secure, SameSite session and remember cookies automatically.

5. Technical Stack

Layer	Technology	Purpose
Front end	HTML5, CSS3, Bootstrap 5.3, Bootstrap Icons	Responsive pages and components
Client logic	Vanilla JavaScript	API calls, cart, dashboard rendering, validation
Back end	PHP 8.2	REST-style JSON endpoints, sessions, business logic
Database	PostgreSQL 15 on Supabase	Persistent relational data
Database access	PDO with prepared statements	Safe, parameterized queries
File storage	Supabase Storage (production), local filesystem (development)	License and registration documents
Payments	Razorpay (test mode)	UPI, card and net-banking payments
OTP email	SMTP (PHP socket client)	Password-reset one-time passwords
Maps	Leaflet, OpenStreetMap, Google Maps links	Location display and address-first directions
Web server	Apache on Docker (Render), XAMPP locally	Hosting the PHP application
Hosting	Render Docker web service, Singapore region	Public HTTPS deployment
Version control	Git and GitHub	Source control and collaboration
Testing	PHP cURL tests, Playwright, live API checks	API, browser and deployment validation

5.1 Why This Stack

PHP and PostgreSQL are widely available, well documented and suitable for a data-driven web application. Supabase provides managed PostgreSQL and object storage without server administration. Bootstrap keeps the interface consistent and responsive. Razorpay offers a straightforward test-mode integration. Docker makes the runtime reproducible on Render, and Playwright provides reliable end-to-end browser testing.

6. Database Design

The database contains ten tables. The design separates common account data from role-specific profiles and keeps order history independent of current inventory.

Table	Key Data	Purpose
`users`	name, email, password hash, phone, address, city, latitude, longitude, role, org type, verified	Common account data
`distributors`	user_id, license number, license image	Distributor profile
`ngos`	user_id, registration number, registration image	Organization profile
`medicines`	name, category, price, MRP, stock, batch, expiry, shelf life, image, distributor, deleted, low-stock flag	Inventory
`orders`	organization, distributor, delivery address, delivery mobile, coordinates, total, status, tracking number, timestamps	Order header
`order_items`	order, medicine, quantity, price	Order line items
`transactions`	order, amount, method, status, gateway reference	Payment record
`notifications`	user, type, text, read status, timestamp	User notifications
`reviews`	order, user, rating, comment	Delivered-order feedback
`audit_logs`	user, action, timestamp	Activity trail

6.1 Relationships

- One user has at most one distributor profile or one organization profile.
- One distributor owns many medicines.
- One organization places many orders; one order belongs to one distributor.
- One order contains many order items and has one transaction.
- One delivered order can have one review.

6.2 Design Notes

Medicine rows use a soft-delete flag because order items reference them. Report and dashboard queries filter on `deleted = 0`, so removed medicines do not inflate live totals. Prices are copied into order items at the time of ordering so later price changes do not alter historical orders. Order totals are computed on the server. Status values are constrained so that only valid lifecycle states can be stored.

The delivery mobile number was added as an additive migration, `ALTER TABLE orders ADD COLUMN IF NOT EXISTS delivery_phone VARCHAR(15)`. It is nullable for historical orders and validated as exactly ten digits for new orders. Rejected registrations are hard-deleted with their

dependent notifications, audit entries, role profile and user row, while license documents are removed through the storage helper.

7. Implementation

7.1 Authentication and Role Management

Registration collects the user's name, email, password, phone, address, city, coordinates, role and, for distributors and organizations, license or registration details and an uploaded document. Phone numbers are normalised to digits and must contain exactly ten digits. Passwords are stored with `password_hash()` and verified with `password_verify()`. A successful login regenerates the session ID and stores the user's identity in the PHP session. Protected endpoints call a shared role check, and the dashboard hides tabs that are not allowed for the current role.

Sign-in is persistent for thirty days. Alongside the session cookie, the server issues a signed `medred_remember` cookie containing the user, password-hash fingerprint and expiry. The value is protected with an HMAC derived from `RESET_SECRET`, marked HttpOnly and SameSite=Lax, and is only issued over HTTPS as Secure. `current_user()` can restore a session from this cookie, while logout, expiry or a password change invalidates it. This keeps users signed in across browser restarts without storing a password or a reusable server-side token.

7.2 Admin Verification and Documents

New distributor and organization accounts remain unverified until an administrator approves them. The verification page lists pending accounts and shows a thumbnail of the uploaded document, which opens the full image in a new tab. Approve and reject actions are written to the audit log. Rejection is destructive by design: after confirming the action, the server removes the pending user's notifications, audit entries, distributor or organization profile and user row, and deletes the uploaded license document through the storage helper. Rejected accounts therefore disappear from the pending queue and cannot be approved accidentally later.

7.3 Medicine Inventory

Distributors add medicines with name, category, price, MRP, stock, batch number, expiry date, shelf life and image. Existing medicines can be edited. Removal uses a soft-delete flag so past orders and invoices remain intact. When stock reaches zero, the medicine is removed from the store and the distributor is notified. Expired medicines and out-of-stock medicines are also removed automatically when the store is loaded, and a low-stock notification is generated below the configured threshold.

7.4 Store, Cart and Geolocation

The store shows available medicines with distributor name, category, price, stock, expiry and shelf life. Users can search by name and filter by category, remaining shelf life and distributor. The cart is stored in the browser so it survives page reloads. Checkout now captures a delivery mobile number immediately after the delivery address. The input accepts digits only, enforces ten digits on the client and on the server, and remembers the last value in a cookie so repeat orders are faster. Delivery location is captured through the browser geolocation API and stored in a cookie; if permission is denied, checkout proceeds with the delivery address and mobile number alone.

7.5 Order Management

When an order is placed, the server validates each medicine, checks expiry and stock, deducts the ordered quantity, groups items by distributor and creates one order per distributor. The delivery mobile number is validated and stored in `orders.delivery_phone`; invalid numbers are rejected before any stock is touched. The minimum order value is ₹500 per distributor. The order, its line items and its transaction are written inside a database transaction. Organizations can cancel an order while it is Pending or Pending Payment; the cancellation restores reserved stock and marks the transaction as Cancelled. Distributors can reject pending orders, which also restores stock.

Each order row and the delivery map display the saved mobile number. The Navigate action opens Google Maps with the delivery address as the destination, and uses stored latitude and longitude only when the address is missing. This avoids relying on geocoding services and lets the driver use the address the organization actually entered.

7.6 Notifications, Reviews and Reports

Notifications are stored in the database with a type, text and read status. They are generated for new orders, payment confirmation, acceptance, rejection, cancellation, delivery, low stock and expiry. The navigation bar shows an unread-count badge refreshed every 60 seconds, and the dashboard refreshes its own badge every 30 seconds after the page loads. Organizations can review a delivered order once, with a rating and an optional comment. Reports show revenue, order counts, medicine counts and stock distribution, with CSV export. All report totals, category breakdowns and low-stock or expiry-risk counts filter out soft-deleted medicines, so administrators see active inventory only. The audit log records sensitive actions with the user and timestamp.

7.7 Module-to-File Mapping

Module	Main Files
Authentication	`auth/login.php`, `logout.php`, `session.php`, `register.php`, `config/db.php`
Password reset	`forgot-password.html`, `auth/forgot.php`
Dashboard	`dashboard.html`, `js/app.js`
Medicine inventory	`api/medicines.php`
Store and cart	`index.html`, `js/app.js`, `js/store.js`
Orders	`api/orders.php`, `js/cart.js`
Payments	`api/payment.php`
Notifications	`api/notifications.php`
Verification	`api/verify.php`, `config/storage.php`
Reports and audit	`api/reports.php`, `api/audit.php`
Reviews and profile	`api/reviews.php`, `api/profile.php`

Module	Main Files
Map and delivery	`map.html`, `delivery-map.html`, `api/locations.php`
Deployment	`Dockerfile`, `docker/`, `health.php`, `scripts/migrate-uploads.php`

7.8 Password Reset over SMTP

The reset flow is implemented without a third-party authentication service. A six-digit OTP is generated on the server, stored as a short-lived hash associated with the account, and sent by the built-in SMTP client. Verification is limited to five attempts, and a successful check issues a signed, expiring reset token. In development, when SMTP is not configured, the OTP is written to a temporary log instead of being emailed; in production, a missing SMTP configuration returns a clear service-unavailable response rather than silently failing.

7.9 Branding and Interface

The product is branded MEDRED throughout, with the line “Don't waste medicines, redistribute hope.” The home page uses the new hero image and logo, the navigation and footer carry the same identity, and page titles, favicons, invoices, emails and error messages were updated consistently. The changes preserve the existing layout and workflows; no feature was removed or redesigned.

8. Order Lifecycle & Payments

8.1 Status Flow

```
Razorpay:  Pending Payment --(verified payment)--> Pending
Cash:      Pending (immediately)

Pending --(distributor accepts)--> Accepted --> Packed --> Shipped
        --> Out for Delivery --> Delivered

Pending / Pending Payment --(organization cancels)--> Cancelled
Pending --(distributor rejects)--> Cancelled
```

Stock is deducted when the order is created. If an order is cancelled or rejected, the stock is restored. When the last units of a medicine are ordered, the medicine is marked removed and the distributor is notified. An unpaid Razorpay order cannot be accepted by the distributor, so payment confirmation is enforced on the server.

8.2 Online Payments

The server creates a Razorpay order with the server-computed total, stores the gateway order reference in the transaction row and returns the public key and order details to the browser. The Razorpay checkout supports UPI, cards and net banking, and UPI can be pre-selected so that the UPI section opens first.

After payment, the browser sends the Razorpay order ID, payment ID and signature to the server. The server recomputes the HMAC-SHA256 signature with the secret key and compares it using a timing-safe comparison. Only a valid signature moves the orders from Pending Payment to Pending and marks the transaction as Completed.

8.3 Cash on Delivery

Cash orders are confirmed immediately with a Pending transaction. The distributor can mark the payment as received after delivery. Payment status is displayed separately from order status so that the two concerns are not confused.

9. Security

Security was treated as a cross-cutting concern rather than an add-on. The main measures are:

- passwords hashed with `password_hash()` and verified with `password_verify()`;
- prepared statements for every database query to prevent SQL injection;
- role checks on every protected endpoint, not only in the interface;
- session ID regeneration after login;
- HMAC-SHA256 verification of Razorpay payments with a timing-safe comparison;
- HMAC-signed, expiring password-reset tokens;
- a signed, HttpOnly, SameSite=Lax, Secure-on-HTTPS remember cookie whose signature covers the user, expiry and password-hash fingerprint;
- generic responses for unknown email addresses to prevent account enumeration;
- a five-attempt limit on OTP verification and single-use OTP records;
- SMTP credentials supplied only through server-side configuration;
- rejected-registration cleanup that removes the user, role profile, notifications and audit rows and deletes the uploaded document;
- `.htaccess` denies for the configuration and log folders;
- PHP execution blocked inside the uploads folder;
- credentials stored in environment variables or `config/secrets.php`, both excluded from Git;
- user-uploaded license documents and logs excluded from Git;
- audit logging of sensitive actions.

The password-reset flow sends a six-digit OTP through the application's SMTP client, verifies it on the server, and issues a short-lived reset token. The new password is hashed before storage. The OTP is never returned by the API; only the development fallback writes it to a local temporary log when SMTP is absent.

10. Performance Optimization

Because the database is hosted remotely, a cold database round trip costs hundreds of milliseconds to several seconds. The following optimizations were applied:

- **Short-lived file cache:** read-heavy GET endpoints cache JSON responses for 10–20 seconds. Every POST clears the cache, so writes are visible immediately.
- **Lazy database connection:** endpoints check the cache before opening a database connection, so cache hits avoid the database entirely.
- **Session lock release:** the PHP session is closed after reading the current user, allowing parallel requests.
- **Combined queries:** report statistics were reduced from four queries to one.
- **Prefetching:** the dashboard warms the APIs for all tabs after login, so tab switches hit the client or server cache.
- **Client-side cache:** the dashboard keeps GET responses for 30 seconds and the store keeps them longer.

Endpoint	Cold response	Cached response
Orders	~916 ms	5–32 ms
Medicines	~1403 ms	3–4 ms
Notifications	~797 ms	27–28 ms
Reports	~1585 ms	12–27 ms

The results show that caching and lazy connections remove most repeated database latency while short TTLs and write invalidation keep data fresh.

11. Testing & Results

Testing was performed at three levels: API tests using PHP and cURL, browser tests using Playwright, and a role-based smoke test that visited every page and tab.

#	Test	Expected Result	Status
1	Login with valid credentials	Session created and dashboard opens	Pass
2	Login with wrong password	401 Invalid email or password	Pass
3	Unverified account login	403 Account pending verification	Pass
4	Admin approves a distributor with a license image	Account verified; document visible	Pass
5	Distributor adds a medicine	Medicine appears in the store	Pass
6	Distributor removes a medicine	Medicine disappears; order history remains	Pass
7	Organization places a cash order	Order Pending; stock reduced	Pass
8	Organization places a Razorpay order	Pending Payment; distributor cannot accept	Pass
9	Razorpay payment with a valid signature	Order Pending; transaction Completed	Pass
10	Razorpay payment with an invalid signature	Payment rejected	Pass
11	Organization cancels a pending order	Cancelled; stock restored; transaction Cancelled	Pass
12	Organization tries to cancel an accepted order	409 error	Pass
13	Ordering the last units of a medicine	Medicine removed; distributor notified	Pass
14	Expired medicine in the store	Medicine removed; distributor notified	Pass
15	Unread notifications	Badge shows the count	Pass
16	Review on a delivered order	Review saved; second review rejected	Pass
17	Forgot password with a registered email	OTP generated and verified	Pass
18	Password reset with a valid token	Password updated; login works	Pass
19	Reuse of a used OTP	Rejected	Pass
20	Tab access outside the user's role	Tab hidden; API returns 403	Pass
21	Admin dashboard totals after soft-deleting medicines	Active totals exclude deleted rows; all-time total remains available	Pass
22	Admin rejects a pending registration	User, profile, notifications, audit rows and uploaded document removed	Pass
23	Checkout with a nine-digit or alphabetic mobile number	422 response; no order or stock change	Pass

#	Test	Expected Result	Status
24	Checkout with a valid ten-digit mobile number	Number stored on the order and shown in dashboard and map	Pass
25	Navigate to an order with an address and coordinates	Google Maps opens with the address as destination	Pass
26	Return to the app after the browser session ends	Signed remember cookie restores the session; logout invalidates it	Pass
27	Password reset request and OTP verification in development	OTP written to the temporary log; valid token issued	Pass

All API and UI test suites passed. The smoke test rendered every role's tabs with zero JavaScript errors; the only console messages were expected 401 responses for guests. Live API checks on the Render deployment confirmed the health endpoint, database connection, authentication and report totals. Production email delivery remains dependent on SMTP credentials being configured in the hosting environment. Test data was removed after the runs.

12. Limitations & Future Work

12.1 Limitations

- Email delivery requires SMTP credentials in the hosting environment; SMS remains a stub.
- District, state and PIN code are not yet separate structured fields.
- Razorpay is used in test mode; live payments require live keys and compliance checks.
- There is no integration with courier or delivery-partner APIs.
- Geolocation depends on browser permission and may be inaccurate; navigation therefore prefers the typed address.
- The application is a responsive web app, not a native mobile app.
- The hosted database and free Render instance add latency to cold requests and may spin down when idle.
- Admin verification is manual, and the interface is available in English only.

12.2 Future Work

- Provide production SMTP credentials and enable live Razorpay payments.
- Integrate an SMS gateway such as Twilio or MSG91.
- Add delivery-partner tracking and route optimization.
- Add barcode or QR scanning for medicine batches.
- Use OCR to assist license verification.
- Add demand forecasting and early expiry alerts.
- Build a mobile application for distributors and field staff.
- Add multi-language support, accessibility improvements, backups and monitoring.

13. Conclusion & References

13.1 Conclusion

MEDRED demonstrates how a focused web application can reduce medicine wastage and improve access for social organizations. The project covers the complete operational cycle: registration, verification, inventory, store, cart, delivery details, payment, order tracking, cancellation, notifications, reviews, reports, persistent sign-in and password recovery.

The implementation emphasizes correctness at trust boundaries: verified accounts, complete cleanup of rejected registrations, server-side payment verification, transactional stock handling, role-based authorization, signed sessions and audit logging. Reports now reflect active inventory only, delivery navigation follows the address the organization entered, and the Docker image runs consistently on Render. Caching and lazy database access keep the interface responsive despite a hosted database. The system is ready for extension with production SMTP credentials, live payment credentials and delivery-partner integration.

13.2 References

1. PHP Documentation — https://www.php.net/docs.php
2. PostgreSQL Documentation — https://www.postgresql.org/docs/
3. Supabase Documentation — https://supabase.com/docs
4. Razorpay Payments Documentation — https://razorpay.com/docs/
5. Bootstrap 5 Documentation — https://getbootstrap.com/docs/5.3/
6. MDN Web Docs — https://developer.mozilla.org/
7. OWASP Application Security Verification Standard — https://owasp.org/www-project-application-security-verification-standard/
8. Playwright Documentation — https://playwright.dev/
9. Leaflet Documentation — https://leafletjs.com/
10. OpenStreetMap — https://www.openstreetmap.org/

Appendix — Setup & API Reference

A. Setup Instructions

1. Install XAMPP with PHP 8.2 and enable the `pdo_pgsql`, `curl` and `fileinfo` extensions.
2. Import `database.sql` into a PostgreSQL or Supabase database. Existing databases need `ALTER TABLE orders ADD COLUMN IF NOT EXISTS delivery_phone VARCHAR(15);`
3. Copy `config/secrets.example.php` to `config/secrets.php` and fill in the database, Razorpay, Supabase and SMTP values.
4. Place the project under the Apache document root and open `index.html`.
5. For password-reset email, set `SMTP_HOST`, `SMTP_PORT`, `SMTP_SECURE`, `SMTP_USER`, `SMTP_PASS` and `SMTP_FROM`. Without SMTP, development writes the OTP to `%TEMP%/medred_otp.log`; production returns a configuration error.
6. For Docker, run `docker compose up --build` with `.env` present, or deploy the repository as a Docker web service on Render and add the same environment variables in the dashboard.
7. Set the Render health check path to `/health.php` and confirm the service reports a healthy database connection.

B. Seed Accounts

All seed accounts use the password `admin123`.

Role	Email
Administrator	admin@medshare.com
Distributor	medplus@medshare.com
Distributor	pharmalink@medshare.com
Organization	hope@medshare.com
Organization	citycare@medshare.com

C. Configuration Keys

Values are supplied as environment variables in Docker and Render, or through `config/secrets.php` locally; neither secrets file is committed.

Variable	Purpose
`DB_DSN`, `DB_USER`, `DB_PASS`	PostgreSQL connection
`SUPABASE_URL`, `SUPABASE_ANON_KEY`, `SUPABASE_SERVICE_ROLE_KEY`	Supabase Storage for uploaded documents
`RAZORPAY_KEY_ID`, `RAZORPAY_KEY_SECRET`	Payment gateway credentials

Variable	Purpose
`RESET_SECRET`	Signing key for reset tokens and remember cookies
`SMTP_HOST`, `SMTP_PORT`, `SMTP_SECURE`, `SMTP_USER`, `SMTP_PASS`, `SMTP_FROM`	Password-reset and notification email
`MEDRED_EMAIL`	Enables notification email when set to true
`APP_ENV`	Set to `production` on Render; enables the OTP log fallback only in development

D. API Endpoints

Endpoint	Purpose
`auth/login.php`, `auth/logout.php`, `auth/session.php`	Authentication
`auth/register.php`	Registration with document upload
`auth/forgot.php`	Password-reset OTP request, verification and reset
`api/medicines.php`	Medicine list, details, add, edit, remove
`api/orders.php`	Order creation with delivery mobile, status actions, cancellation, list
`api/payment.php`	Razorpay order creation and verification
`api/notifications.php`	List and mark notifications as read
`api/verify.php`	Admin approval, or rejection with complete registration cleanup
`api/distributors.php`	Distributor profiles and admin list
`api/reviews.php`	Reviews for delivered orders
`api/reports.php`	Statistics and CSV export
`api/audit.php`	Audit log
`api/profile.php`	Profile update
`api/locations.php`	Verified locations for the map
`health.php`	Deployment health check with database connectivity

End of Report